

# Homework 2 - Stat 495 - Fall 2025

Daizy Buluma

2025-10-02

**Homework 2 - Due Wednesday, Oct. 1st by midnight to Gradescope**

## Practicing Academic Integrity

If you worked with others or used resources outside of provided course material, such as Generative AI (anything besides our textbook, course materials in the repo, labs, R help menu) to help complete this assignment, please acknowledge them below using a bulleted list. Remember that according to our class framework, if you engage with Generative AI tools, the tool and date of use must be listed here, and then a complete record of the chat history must be appended to the assignment.

*I acknowledge the following individuals with whom I worked on this assignment:*

Name(s) and how they assisted you

- 

*I used the following sources to help complete this assignment:*

Source(s) and where used in your submission

-

## PROBLEMS TO TURN IN: Add 1 and Add 2

### Additional 1 - Applications to College

Adapted from similar problems in ISLR

```
data(College)
```

This data set contains information from 1995 on US Colleges from US News and World Report (see help file for details). Our goal is to predict the number of applications received (Apps) using the other variables as potential predictors.

part a: Split the data into training and test data sets. What % went into your training set and why?

SOLUTION: I chose the common 70:30 split where 70% went to the training set and 30% was the testing set. This split ensures that the model has enough data to sufficiently learn the patterns but also have a reasonable chunk left to test the model so that it can generalize well on unseen data.

```
set.seed(495)
split_app <- initial_split(College, prop = 0.7,
                           strata = "Apps")
train_app <- training(split_app)
test_app  <- testing(split_app)
```

part b: Set up a cross-validation framework to use for comparing models below. Even models that don't have tuning parameters to pick go through the framework to enable later comparisons.

SOLUTION:

```
#Repeated cv because n=777 < 10000
myControl <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
```

part c: Fit a “kitchen sink” linear regression model on the training set. Display the final model coefficients.

SOLUTION:

```

set.seed(495)
reglm_model1 <- train(
  Apps ~ .,
  data = train_app,
  method = "lm",
  trControl = myControl
)

summary(reglm_model1)

```

Call:

```
lm(formula = .outcome ~ ., data = dat)
```

Residuals:

|       |      |        |     |      |
|-------|------|--------|-----|------|
| Min   | 1Q   | Median | 3Q  | Max  |
| -5133 | -447 | -35    | 347 | 7340 |

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t ) |     |
|-------------|-----------|------------|---------|----------|-----|
| (Intercept) | -2.25e+02 | 5.21e+02   | -0.43   | 0.66639  |     |
| PrivateYes  | -4.76e+02 | 1.74e+02   | -2.74   | 0.00632  | **  |
| Accept      | 1.62e+00  | 4.78e-02   | 33.95   | < 2e-16  | *** |
| Enroll      | -6.72e-01 | 2.41e-01   | -2.79   | 0.00553  | **  |
| Top10perc   | 6.23e+01  | 6.95e+00   | 8.96    | < 2e-16  | *** |
| Top25perc   | -1.89e+01 | 5.56e+00   | -3.41   | 0.00071  | *** |
| F.Undergrad | -1.20e-02 | 4.23e-02   | -0.28   | 0.77613  |     |
| P.Undergrad | 7.88e-02  | 3.80e-02   | 2.07    | 0.03871  | *   |
| Outstate    | -1.01e-01 | 2.43e-02   | -4.18   | 3.4e-05  | *** |
| Room.Board  | 1.80e-01  | 5.97e-02   | 3.01    | 0.00274  | **  |
| Books       | -1.08e-01 | 2.92e-01   | -0.37   | 0.71170  |     |
| Personal    | -3.53e-03 | 7.93e-02   | -0.04   | 0.96448  |     |
| PhD         | -9.35e+00 | 5.72e+00   | -1.64   | 0.10260  |     |
| Terminal    | -2.27e+00 | 6.38e+00   | -0.36   | 0.72213  |     |
| S.F.Ratio   | 1.64e+01  | 1.62e+01   | 1.01    | 0.31106  |     |
| perc.alumni | 1.36e+00  | 5.29e+00   | 0.26    | 0.79658  |     |
| Expend      | 6.24e-02  | 1.45e-02   | 4.30    | 2.0e-05  | *** |
| Grad.Rate   | 7.30e+00  | 3.67e+00   | 1.99    | 0.04715  | *   |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1100 on 523 degrees of freedom

Multiple R-squared: 0.929, Adjusted R-squared: 0.926  
F-statistic: 400 on 17 and 523 DF, p-value: <2e-16

part d: Fit a linear regression model using an automated variable selection method of your choice (from Stat 230) on the training set. Display the final model coefficients.

SOLUTION:

```
set.seed(495)
steplm_model1 <- train(
  Apps ~ .,
  data = train_app,
  method = "lmStepAIC",
  trace = FALSE,
  trControl = myControl
)

steplm_model1
```

Linear Regression with Stepwise Selection

541 samples  
17 predictor

No pre-processing  
Resampling: Cross-Validated (10 fold, repeated 10 times)  
Summary of sample sizes: 485, 486, 487, 486, 486, 487, ...  
Resampling results:

| RMSE   | Rsquared | MAE     |
|--------|----------|---------|
| 1166.5 | 0.914489 | 676.464 |

```
summary(steplm_model1)
```

Call:

```
lm(formula = .outcome ~ PrivateYes + Accept + Enroll + Top10perc +  
    Top25perc + P.Undergrad + Outstate + Room.Board + PhD + Expend +  
    Grad.Rate, data = dat)
```

Residuals:

| Min   | 1Q   | Median | 3Q  | Max  |
|-------|------|--------|-----|------|
| -5085 | -447 | -38    | 340 | 7387 |

Coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t ) |     |
|-------------|-----------|------------|---------|----------|-----|
| (Intercept) | -35.2884  | 331.8801   | -0.11   | 0.91536  |     |
| PrivateYes  | -486.3887 | 166.1671   | -2.93   | 0.00357  | **  |
| Accept      | 1.6219    | 0.0468     | 34.63   | < 2e-16  | *** |
| Enroll      | -0.7230   | 0.1343     | -5.38   | 1.1e-07  | *** |
| Top10perc   | 62.3654   | 6.8589     | 9.09    | < 2e-16  | *** |
| Top25perc   | -19.0770  | 5.4777     | -3.48   | 0.00054  | *** |
| P.Undergrad | 0.0743    | 0.0366     | 2.03    | 0.04271  | *   |
| Outstate    | -0.1014   | 0.0232     | -4.37   | 1.5e-05  | *** |
| Room.Board  | 0.1707    | 0.0577     | 2.96    | 0.00323  | **  |
| PhD         | -10.4722  | 3.8552     | -2.72   | 0.00682  | **  |
| Expend      | 0.0563    | 0.0133     | 4.24    | 2.6e-05  | *** |
| Grad.Rate   | 7.6045    | 3.5136     | 2.16    | 0.03089  | *   |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1090 on 529 degrees of freedom

Multiple R-squared: 0.928, Adjusted R-squared: 0.927

F-statistic: 623 on 11 and 529 DF, p-value: <2e-16

part e: Fit a ridge regression model on the training set, with lambda chosen by cross-validation. Report the lambda chosen, and display the final model coefficients.

SOLUTION: Lambda = 305.386

```
myGrid <- expand.grid(alpha = 0,
                      lambda = 10^seq(10, -2, length = 100))

set.seed(495)
ridge_model1 <- train(
  Apps ~ .,
  data = train_app,
  method = "glmnet",
  tuneGrid = myGrid,
  trControl = myControl
)
```

Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,

: There were missing values in resampled performance measures.

```
# ridge_model1  
# To get the best lambda  
ridge_model1$bestTune$lambda
```

```
[1] 305.386
```

```
# to get the coefficients for the final model  
coef(ridge_model1$finalModel, s = ridge_model1$bestTune$lambda)
```

18 x 1 sparse Matrix of class "dgCMatrix"

```
          s0  
(Intercept) -1.33325e+03  
PrivateYes   -5.15080e+02  
Accept       1.05286e+00  
Enroll       4.20030e-01  
Top10perc    3.31962e+01  
Top25perc    -1.55855e+00  
F.Undergrad  4.10586e-02  
P.Undergrad  4.69775e-02  
Outstate     -3.75674e-02  
Room.Board   2.14006e-01  
Books        7.19456e-02  
Personal     -2.94090e-02  
PhD          -3.55456e+00  
Terminal     -5.40545e+00  
S.F.Ratio    1.16750e+01  
perc.alumni  -7.88352e+00  
Expend       6.83830e-02  
Grad.Rate    1.13606e+01
```

part f: Fit a lasso model on the training set, with lambda chosen by cross-validation.  
Report the lambda chosen, and display the final model coefficients.

SOLUTION: Lambda = 24.7708

```
myGrid2 <- expand.grid(alpha = 1,  
                      lambda = 10^seq(10, -2, length = 100))  
  
set.seed(495)
```

```
lasso_model1 <- train(
  Apps ~ .,
  data = train_app,
  method = "glmnet",
  tuneGrid = myGrid2,
  trControl = myControl
)
```

Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,  
: There were missing values in resampled performance measures.

```
# to get the best lambda
lasso_model1$bestTune$lambda
```

```
[1] 24.7708
```

```
# to get the coefficients for the final model
coef(lasso_model1$finalModel, s = lasso_model1$bestTune$lambda)
```

18 x 1 sparse Matrix of class "dgCMatrix"

```
      s0
(Intercept) -456.8461559
PrivateYes   -357.1098327
Accept       1.4930067
Enroll       -0.3078041
Top10perc    44.6746976
Top25perc    -7.3679433
F.Undergrad  .
P.Undergrad  0.0309715
Outstate     -0.0642134
Room.Board   0.1387772
Books        .
Personal     .
PhD          -5.5794984
Terminal     -2.6239453
S.F.Ratio    0.1074033
perc.alumni  .
Expend       0.0535086
Grad.Rate    4.0307869
```

part g: Now we need to work to compare the models. We want to compare both the test errors and coefficients, at a minimum. You can also compare their results on the training set or through the CV framework. You'll need to do some work before addressing the following questions as part of your response. Do the test errors differ much? (Go compute them if you didn't above and show your work.) Do the coefficients differ greatly? In particular, if any variables were left out of the model in (d) or (f), is there any insight that they might have been removed based on the models in (c) or (e)?

SOLUTION:

```
summary(resamples(list(
  lm_model = reglm_model1,
  step_model = stepAIC_model1,
  ridge_model = ridge_model1,
  lasso_model = lasso_model1
)))
```

Call:

```
summary.resamples(object = resamples(list(lm_model = reglm_model1, step_model
  = stepAIC_model1, ridge_model = ridge_model1, lasso_model = lasso_model1)))
```

Models: lm\_model, step\_model, ridge\_model, lasso\_model

Number of resamples: 100

MAE

|             | Min.    | 1st Qu. | Median  | Mean    | 3rd Qu. | Max.     | NA's |
|-------------|---------|---------|---------|---------|---------|----------|------|
| lm_model    | 522.843 | 618.433 | 665.503 | 684.870 | 730.105 | 959.974  | 0    |
| step_model  | 510.643 | 612.084 | 658.466 | 676.464 | 719.116 | 956.979  | 0    |
| ridge_model | 489.873 | 584.839 | 657.909 | 687.243 | 725.771 | 1114.787 | 0    |
| lasso_model | 475.438 | 573.910 | 632.406 | 652.705 | 698.806 | 978.560  | 0    |

RMSE

|             | Min.    | 1st Qu. | Median  | Mean    | 3rd Qu. | Max.    | NA's |
|-------------|---------|---------|---------|---------|---------|---------|------|
| lm_model    | 690.505 | 923.646 | 1053.99 | 1181.95 | 1290.12 | 2197.53 | 0    |
| step_model  | 676.763 | 921.037 | 1037.67 | 1166.50 | 1279.36 | 2186.85 | 0    |
| ridge_model | 652.111 | 881.135 | 1027.61 | 1243.33 | 1240.18 | 3125.08 | 0    |
| lasso_model | 624.902 | 886.008 | 1020.87 | 1165.01 | 1289.07 | 2257.46 | 0    |

Rsquared

|  | Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. | NA's |
|--|------|---------|--------|------|---------|------|------|
|--|------|---------|--------|------|---------|------|------|



```
lm_model      0.799857 0.884234 0.929322 0.912774 0.942183 0.969049    0
step_model    0.804198 0.884938 0.930699 0.914489 0.944280 0.970376    0
ridge_model   0.802635 0.881898 0.920162 0.909917 0.937109 0.972348    0
lasso_model   0.812272 0.890684 0.931271 0.915639 0.944392 0.970645    0
```

The test errors differ minimally across the 4 models. By MAE, RMSE and R-squared we might prefer lasso.

```
reglm_coefs <- summary(reglm_model1)$coefficients[,1]
ridge_coefs <- coef(ridge_model1$finalModel, s = ridge_model1$bestTune$lambda)[,1]
lasso_coefs <- coef(lasso_model1$finalModel, s = lasso_model1$bestTune$lambda)[,1]

steplm_coefs <- summary(steplm_model1)$coefficients[,1]

var_names <- names(reglm_coefs)
steplm_alignedcoefs <- setNames(numeric(length(var_names)), var_names)
common_terms <- intersect(var_names, names(steplm_coefs))
steplm_alignedcoefs[common_terms] <- steplm_coefs[common_terms]

mydata <- data.frame(cbind(reglm_coefs, steplm_alignedcoefs, ridge_coefs, lasso_coefs))

mydata <- mydata |>
  mutate(name = row.names(mydata)) |>
  rename(steplm_coefs = steplm_alignedcoefs)

mydata |>
  select(reglm_coefs, steplm_coefs, ridge_coefs, lasso_coefs) |>
  kable(booktabs = TRUE)
```

|             | reglm_coefs | steplm_coefs | ridge_coefs  | lasso_coefs |
|-------------|-------------|--------------|--------------|-------------|
| (Intercept) | -224.670322 | -35.288447   | -1333.248851 | -456.846156 |
| PrivateYes  | -475.889553 | -486.388726  | -515.079873  | -357.109833 |
| Accept      | 1.622051    | 1.621920     | 1.052861     | 1.493007    |
| Enroll      | -0.671587   | -0.723014    | 0.420030     | -0.307804   |
| Top10perc   | 62.310362   | 62.365421    | 33.196235    | 44.674698   |
| Top25perc   | -18.942829  | -19.077023   | -1.558550    | -7.367943   |
| F.Undergrad | -0.012024   | 0.000000     | 0.041059     | 0.000000    |
| P.Undergrad | 0.078783    | 0.074256     | 0.046977     | 0.030972    |

|             | reglm_coefs | steplm_coefs | ridge_coefs | lasso_coefs |
|-------------|-------------|--------------|-------------|-------------|
| Outstate    | -0.101459   | -0.101433    | -0.037567   | -0.064213   |
| Room.Board  | 0.179553    | 0.170681     | 0.214006    | 0.138777    |
| Books       | -0.108052   | 0.000000     | 0.071946    | 0.000000    |
| Personal    | -0.003533   | 0.000000     | -0.029409   | 0.000000    |
| PhD         | -9.347977   | -10.472226   | -3.554556   | -5.579498   |
| Terminal    | -2.269162   | 0.000000     | -5.405449   | -2.623945   |
| S.F.Ratio   | 16.423373   | 0.000000     | 11.675041   | 0.107403    |
| perc.alumni | 1.363911    | 0.000000     | -7.883518   | 0.000000    |
| Expend      | 0.062347    | 0.056250     | 0.068383    | 0.053509    |
| Grad.Rate   | 7.304105    | 7.604512     | 11.360557   | 4.030787    |

Lasso removes 4 variables while the step wise regression model drops 6 variables. There doesn't appear to any important insight lost as a result of the removal of those variables.

```
#Using the test set
regtest_pred <- predict(reglm_model1, test_app)
steplm_pred <- predict(steplm_model1, test_app)
ridgetest_pred <- predict(ridge_model1, test_app, s = ridge_model1$bestTune$lambda)
lassotest_pred <- predict(lasso_model1, test_app, s = lasso_model1$bestTune$lambda)
# Compute MSEs
regtest_mse <- mean((test_app$Apps - regtest_pred)^2)
regtest_mse
```

```
[1] 878963
```

```
ridgetest_mse <- mean((test_app$Apps - ridgetest_pred)^2)
ridgetest_mse
```

```
[1] 862963
```

```
steplmtest_mse <- mean((test_app$Apps - steplm_pred)^2)
steplmtest_mse
```

```
[1] 869958
```

```
lassotest_mse <- mean((test_app$Apps - lassotest_pred)^2)
lassotest_mse
```

[1] 827186

Lasso still performing better than the other models.

part h: Which final model would you select here? Why?

SOLUTION: I would select the lasso model since it is the preferred model if we consider the all the test errors. Also, it removed 4 predictors from the model which simplifies interpretation of the final model.

## Additional 2 - Soil predictions

The data comes from a Kaggle competition: <https://www.kaggle.com/c/afsis-soil-properties>. The original data set contained 3600 variables, 3599 possible predictors (really, 3578 and some other variables) and a response, Sand. The 3599 predictors were reduced to 106 that can “best” distinguish between the two levels of Depth (another variable in the data set). The resulting data set of 107 variables (106 predictors and the response variable, Sand) was saved in the data set “newsoil”. The row numbers should be removed as demonstrated below. We want to use this data set due to the large number of predictors to see what our shrinkage methods do.

```
newsoil <- read.csv("https://awagaman.people.amherst.edu/stat495/newsoil.csv",
                    header = T)
newsoil <- select(newsoil, -X)
```

Our focus is on predicting the response variable Sand, using the selected variables from previous work.

part a: Split the data set into a training and test set (75/25), with a seed of your choice.

SOLUTION:

```
set.seed(495)
split_sand <- initial_split(newsoil, prop = 0.75,
                           strata = "Sand")
train_sand <- training(split_sand)
test_sand <- testing(split_sand)
```

part b: Create a cross-validation framework to use for the models that will be fit below.

SOLUTION:

```
#dim(newsoil)
#Repeated cv because n=1157 < 10000
myControl2 <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
```

part c: Fit lasso models to predict Sand using all the possible predictors. Choose two lasso models - one that has a “best” lambda determined in some appropriate way, and another model with a different non-zero lambda of your choice. Report your lambdas. How many slope coefficients are set to 0 in each of your chosen lasso models?

Hint: You only have to train the lasso model once. It saves coefficients for multiple values of lambda. Get the coefficients at the two values of lambda and save them, then count the zero or non-zero entries in the vectors. Note that you aren't required to display these vectors, because of their length.

SOLUTION:

The cross-validation procedure selected a best  $\lambda = 0.01$ . At this  $\lambda$ , 101 slope coefficients were shrunk to zero out of the 106 predictors. At my non-zero  $\lambda = 1$ , the shrinkage was much stronger, and all 106 slope coefficients were set to zero, leaving only the intercept.

```
myGrid3 <- expand.grid(alpha = 1,
                      lambda = 10^seq(10, -2, length = 100))

set.seed(495)
lasso_model2 <- train(
  Sand ~ .,
  data = train_sand,
  method = "glmnet",
  tuneGrid = myGrid3,
  trControl = myControl2
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

```
# to get the best lambda
lasso_model2$bestTune$lambda
```

```
[1] 0.01
```

```
# to get the coefficients for the final model
coef1 <- coef(lasso_model2$finalModel, s = lasso_model2$bestTune$lambda)
coef2 <- coef(lasso_model2$finalModel, s = 1)
```

part d: Fit an elastic net model to predict Sand using all the possible predictors with appropriate searches for the tuning parameters. How many slope coefficients are set to 0 in your elastic net model? What are the chosen values of the tuning parameters? Did it pick a lasso or ridge model? How do you know?

SOLUTION:

The model selected  $\alpha = 0.05$  and  $\lambda = 0.01$  as the optimal tuning parameters. 16 slope coefficients out of 106 were set to zero, while the rest remained nonzero. Because  $\alpha = 0.05$  is very close to zero, the model is much closer to ridge regression than to lasso. I know this because  $\alpha = 1$  corresponds to a pure lasso model and  $\alpha = 0$  corresponds to a pure ridge model. Since  $\alpha = 0.05$  is closer zero, the model behaved like ridge regression, shrinking most slope coefficients but still performing slight variable selection.

```
myGrid4 <- expand.grid(alpha = seq(0, 1, by = 0.05),
                      lambda = 10^seq(10, -2, length = 100))

# This will take some time to run because of all the alpha values AND lambda values
set.seed(495)
enet_model1 <- train(
  Sand ~ .,
  data = train_sand,
  method = "glmnet",
  tuneGrid = myGrid4,
  trControl = myControl2
)
```

Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,  
: There were missing values in resampled performance measures.

```
# to get the best alpha and lambda
enet_model1$bestTune$alpha
```

```
[1] 0.05
```

```
enet_model1$bestTune$lambda
```

```
[1] 0.01
```

```
# to get the coefficients for the final model
coef3 <- coef(enet_model1$finalModel, s = enet_model1$bestTune$lambda)
```

part e: Compute test MSEs for both of your lasso models and your elastic net model.

SOLUTION:

```
# Refit lasso for lambda = 1
myGrid5 <- expand.grid(alpha = 1,
                      lambda = 1)

set.seed(495)
lasso_model22 <- train(
  Sand ~ .,
  data = train_sand,
  method = "glmnet",
  tuneGrid = myGrid5,
  trControl = myControl2
)
```

Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,  
: There were missing values in resampled performance measures.

```
lassotest1_pred <- predict(lasso_model2, test_sand, s = lasso_model2$bestTune$lambda)
lassotest22_pred <- predict(lasso_model22, test_sand)
enettest_pred <- predict(enet_model1, test_sand, s = enet_model1$bestTune$lambda)

#Computing mse
lassotest1_mse <- mean((test_sand$Sand - lassotest1_pred)^2)
lassotest1_mse
```

```
[1] 0.31523
```

```
lassotest22_mse <- mean((test_sand$Sand - lassotest22_pred)^2)
lassotest22_mse
```

```
[1] 1.01401
```

```
enettest1_mse <- mean((test_sand$Sand - enettest_pred)^2)
enettest1_mse
```

```
[1] 0.295979
```

part f: Write a few sentences to address the following questions. Does the test MSE from the model with the “best” lambda suggest it is in fact a better predictive model than your other lasso model? Is the elastic net model better than the lasso models? Which final model would you choose here from these three models? Why?

Hint: You can compare metrics not specifically referenced here, such as performance on the training set or in the CV results for the final model selection. You need to figure out your own rationale for your model choice and there are many options.

SOLUTION:

The test MSE for the lasso model with the “best” lambda is 0.31523, which is much lower than the test MSE for the other lasso model (1.01401). This confirms that the best lambda model is indeed a better predictive model than the alternative lasso model. The elastic net model has the lowest test MSE (0.295979), indicating that it outperforms both lasso models in predictive accuracy.

I would choose the elastic net model as the final model based on these results. It achieves the lowest test error (MSE) and combines the properties of both lasso and ridge regression i.e. balances variable selection with coefficient shrinkage which in turn improves the stability of the prediction.